Oregon State University
College of Engineering

ENGR 103: Engineering Computation and Algorithmic Thinking

Studio #7 – Lists

Introduction

In today's studio, you will complete several tasks that will help you with Homework #5. You should be prepared to engage with your classmates **for the entire studio time**. The main goals for today are:

1. To create a program to practice using lists.
2. To work on Homework #5 in a supportive environment.

The expectations for you today are:

1. Stay engaged with the material for the entire time (take a quick stretch break as needed).
2. Focus more on what you will learn through this process rather than on finishing in time.
3. Remember the things you learned about teamwork, communication, and listening from other classes (i.e., ENGR 100 and 102, if you have taken them).
4. Be respectful of your classmates' input and strive to be part of an inclusive culture in this learning space.

The graduate teaching assistant (GTA) and undergraduate teaching assistants (ULAs) will guide you through today's work. Please follow along with the instructions below. Feel free to ask questions if anything is unclear. **An evaluation rubric for today's studio is included on page 3**.

When you are ready to leave, you **must** show all the work you completed in today's studio to a TA to get credit.

Thank you for your hard work and engagement! If you experience technical difficulties, you must communicate with your GTA and/or ULAs and explain the situation. For help outside of class/studio time, please access the Canvas course's main page to see the **Student Office Hours** schedule for the GTAs and ULAs assigned to this course.

Oregon State University
College of Engineering

Work to be Completed (~ 100 min)

Part 1 (~70 min)

Follow along with the TA for a demo.

As a team, create a slightly modified guessing game program from Studio 5. Write a function that populates a list with 10 random numbers in the range 1 to 100 (refer to Studio 5 for help). Now, write a function that allows the user up to 5 attempts to guess one of the 10 random numbers. If the user guesses one of the 10 numbers correctly in 5 or less attempts, then output that they are a winner along with the contents of the list.

Make sure to begin by doing all of **Polya's problem-solving steps** (**see page 4 of this document**) and develop pseudocode or a flowchart:

- Your program needs to check the user's inputs. Ensure that all numbers entered are integers and are in the range 1 to 100.
- Your program should have at least one function in addition to the **`main()`** function. **More is better!** Aim to keep all functions to 30 lines of code or less.
- **Be sure to test your program**. Is it doing what you expect for **ALL** the possibilities?

Extended Learning 1

If you finish your program, start adding the following:

- Add more informative input and output prompts, such as the guess number they are on and how many more guesses they have.
- On any kind of error, continue to ask the user for that entry number – e.g., if it is attempt 1 and a user enters -1, give the reason for the error and ask again for attempt 1.
- Add information that tells the user if they are getting closer or not compared to the last guess.
- Add information that tells the user which of the 10 numbers they were the closest to.

Extended Learning 2

- Studio 6
  - Finish Studio 6 if you did not finish it last week.
  - Use at least one list in Studio 6.
  - Look back at your functions and see if they could be improved (i.e., modified, combined, or split).
- Homework #3 – Quality Control
  - Modify Homework #3 to include **at least one list**.
- Modify Homework #3 to ask the user for *n* (the sample size) and update the code to accommodate for this.

Part 2 (~30 min)

Work in teams on **Part 1 of Homework #5**. The goal is not necessarily to finish this part of the assignment, but to make progress toward completing it.

1. Open Homework #5 through Canvas. Reread the assignment to make sure you understand everything.
2. Choose an appropriate group based on your progress on Part 1 of Homework #5 thus far. These groups will be divided into (a) those who have not started the assignment at all, (b) those who have done just a little, and (c) those who are almost complete. There might be several groups for each category.
3. Work as a group to help each other develop answers for Step 1, Step 2, and Step 4a. **Use the submission template for Part 1 provided by Dr. Porter to guide your discussion.**
4. If you do not finish the entire Part 1 of Homework #5, that is fine. If you finish it, take this time to help someone else in your group.
5. If your answers to Part 1 of Homework #5 are complete, see if you can work with others in your group to make things better. Do your answers address all the questions about your understanding of the problem? How precise, correct, and understandable is your pseudocode (or flowchart)? Is your test plan incorporating enough good, bad, and edge cases?

Check-Out (~10 min)

Thank you for your hard work and engagement! Be sure to inform the GTA that you are ready to leave to receive credit for today's studio.

Evaluation

Topic	Description	Points
Part 1: Modified guessing game program plan	Create a plan for the modified guessing game with your group using Polya's steps for problem solving and pseudocode or a flow chart.	**4**
Part 1: Modified guessing game program	Create the program. If you finish, add additional items from the extended learning opportunities.	**2**
Part 2: Homework #5	Stay engaged and make progress on Part 1 of Homework #5 with your group.	**2**
Engagement	• Be involved with your group • Contribute to the discussion • Help others whenever you can • Work the entire studio	**2**

Oregon State University
College of Engineering

Polya's Problem-Solving Abbreviated

- **Step 1 – Understanding and Info Gathering**
  - Point of the program and general requirements
  - Inputs, Calculations, and Outputs
  - Assumptions and additional requirements (e.g., error checking, formatting, etc.)
- **Step 2 – Develop a plan of action**
  - Should follow directly from the information in Step 1
  - (Semi-) Detailed Sequence of steps needed (pseudocode or flowchart)
  - Not actually in Python code, but easy to read and easy to go directly into Python
- **Step 3 – Develop a test plan (actual test values and expected output values needed)**
  - How will you know if the program works correctly?
  - What input should work, and what should the result be with that input?
  - What input should not work, and what do you expect the program will do in that case?
  - What inputs (or actions) could cause errors (logical/calculation/syntax/etc.)? i.e., Edge cases